Parte 2: Comunicação MQTT e Funcionamento do Protótipo

A comunicação entre o protótipo simulado e o painel de monitoramento foi implementada utilizando o protocolo MQTT, escolhido por ser um protocolo leve de publicação/assinatura (publish/subscribe), ideal para dispositivos com poucos recursos, como o ESP32 simulado no Wokwi. Nesse modelo, o dispositivo simulado atua como publicador: realiza a leitura periódica dos dados de temperatura do ar e nível de gás por meio dos sensores conectados ao ESP32 e, em seguida, publica essas informações em tópicos MQTT específicos definidos para o projeto — *parquecoco/ponto1/temperatura, parquecoco/ponto1/gas e parquecoco/ponto1/alerta* — estabelecendo assim os canais de comunicação entre os dispositivos. Cada mensagem é transmitida no formato JSON, contendo o identificador do ponto de monitoramento, o valor da leitura e o registro de data e hora, o que padroniza a troca de informações e facilita sua interpretação pelas demais partes do sistema.

Antes da integração completa com o protótipo, foi realizado um teste de publicação e recebimento de mensagens em um broker MQTT público, confirmando o correto funcionamento da comunicação entre os dispositivos e validando os tópicos e o formato de dados definidos para o projeto. Por fim, quando os valores coletados ultrapassam os limites de risco estabelecidos (como temperatura elevada ou concentração anormal de gás), os atuadores simulados — *um LED, representando um alerta visual, e um buzzer, responsável pelo alerta sonoro* — são acionados simultaneamente, e uma mensagem de alerta é publicada no tópico correspondente, permitindo que a equipe de monitoramento do Parque Estadual do Cocó identifique rapidamente o ponto que necessita de verificação, cumprindo assim o objetivo de reduzir o tempo de detecção de possíveis focos de incêndio.

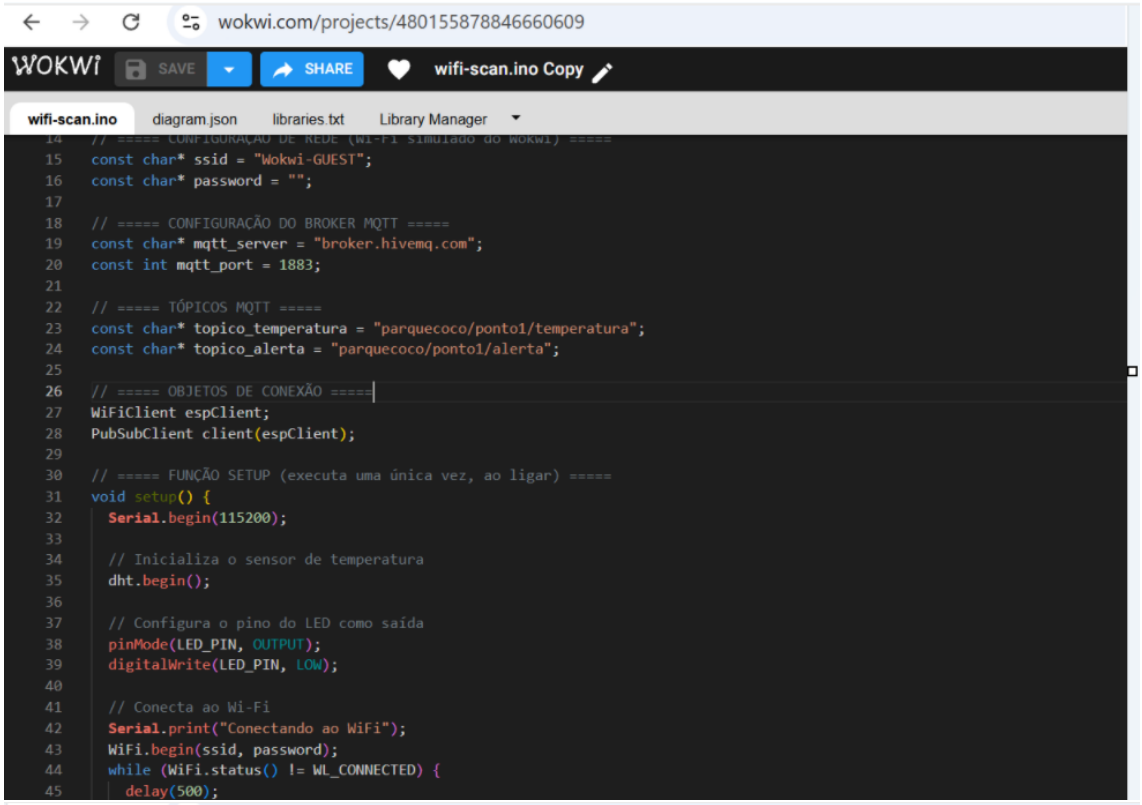
A adição do sinal sonoro reforça a percepção do alerta mesmo quando o painel visual não está sendo observado no momento, aumentando a eficácia da resposta da equipe responsável pelo monitoramento. Os dados publicados nesses tópicos serão posteriormente consumidos por um fluxo em Node-RED, responsável pela visualização em dashboard e pelo registro histórico das leituras.

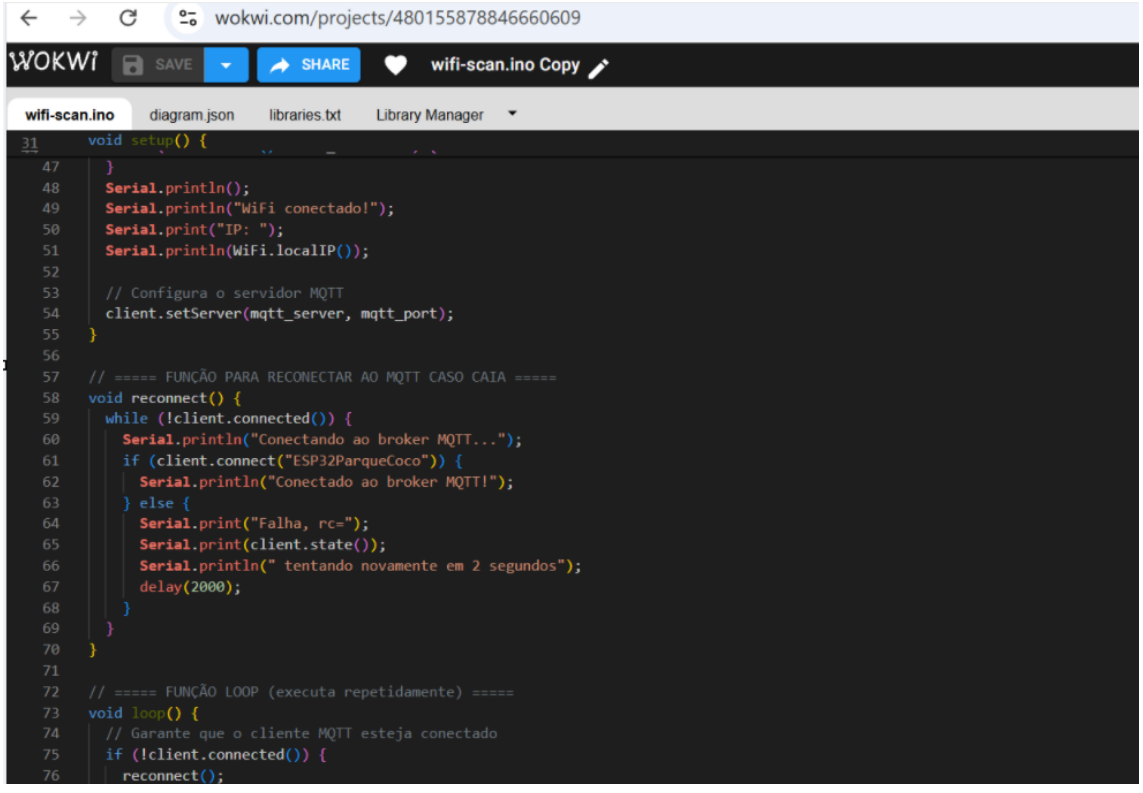
Resultados obtidos: Evidenciados por meio de capturas de tela do ciclo completo executado — publicação das mensagens pelo protótipo, recebimento confirmado no broker MQTT e disparo do alerta visual e sonoro correspondente.

Ciclo percorrido: **código → publicação → broker → recebimento confirmado**

Captura de telas com a configuração e publicação MQTT no protótipo (Wokwi)

Definição do broker, tópicos e JSON





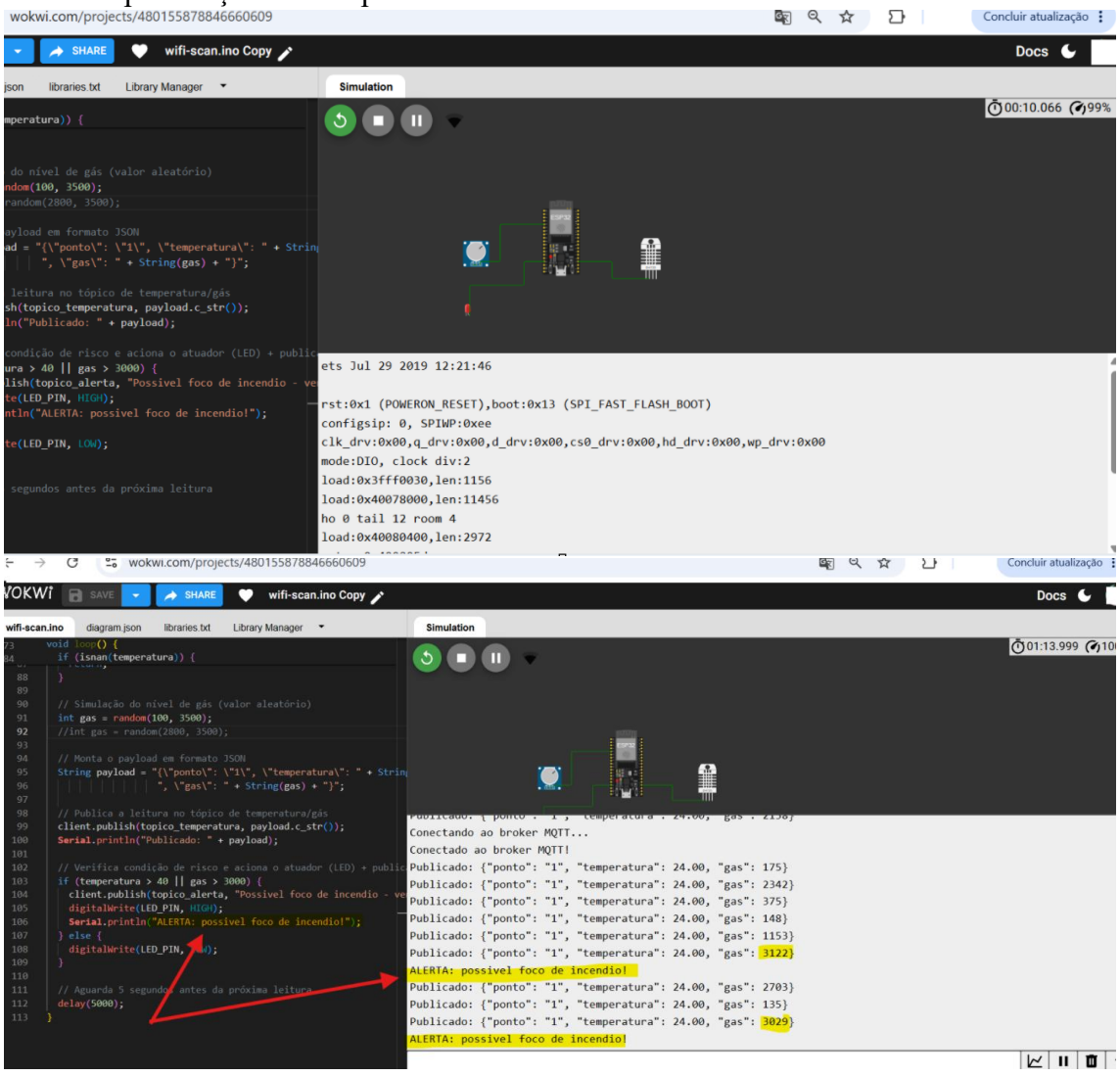
wokwi.com/projects/480155878846660609

WOKWI SAVE SHARE wifi-scan.ino Copy

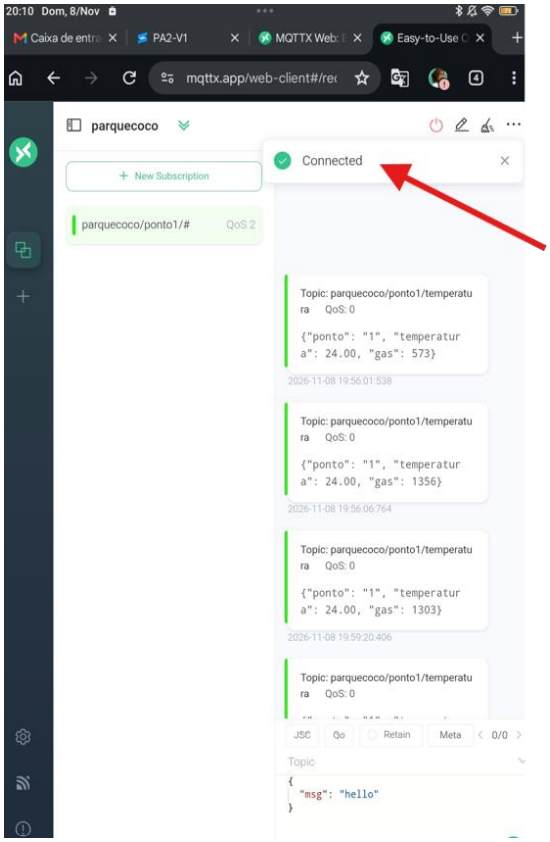
wifi-scan.ino | diagram.json | libraries.txt | Library Manager

```
73    void loop() {
84      if (isnan(temperatura)) {
88      }
89
90      // Simulação do nível de gás (valor aleatório)
91      int gas = random(100, 3500);
92      //int gas = random(2800, 3500);
93
94      // Monta o payload em formato JSON
95      String payload = "{\"ponto\": \"1\", \"temperatura\": " + String(temperatura) +
96                       ", \"gas\": " + String(gas) + "}";
97
98      // Publica a leitura no tópico de temperatura/gás
99      client.publish(topico_temperatura, payload.c_str());
100     Serial.println("Publicado: " + payload);
101
102     // Verifica condição de risco e aciona o atuador (LED) + publica alerta
103     if (temperatura > 40 || gas > 3000) {
104       client.publish(topico_alerta, "Possivel foco de incendio - verificar ponto 1");
105       digitalWrite(LED_PIN, HIGH);
106       Serial.println("ALERTA: possivel foco de incendio!");
107     } else {
108       digitalWrite(LED_PIN, LOW);
109     }
110
111     // Aguarda 5 segundos antes da próxima leitura
112     delay(5000);
113   }
```

Leitura e publicação em tempo real



Conexão bem-sucedida no MQTTX



Mensagem de alerta no MQTTX.

